

Compiler engineer with professional LLVM 22 experience at MCST and a separate x86-64 backend project. I implemented the path from SSA/CFG through liveness/interference, register allocation/spills, and phi lowering to SysV x86-64; generated code runs in a separate process and is compared against a reference interpreter.

2022: Public compiler project · 2023: Language tooling · 2025: Client/product + toolchain · 2026 →: Professional compilers / analysis

| EXPERIENCE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | SKILLS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>2026 — present</div><div><b>ISP RAS — Static Analysis Engineer</b><ul style="list-style-type: none"><li>Contribute to SharpChecker, ISP RAS's C#.NET static-analysis platform.</li></ul></div></div> <div><div>July — August 2026</div><div><b>MCST — Compiler Engineering Intern</b><ul style="list-style-type: none"><li>Implemented a LICM pass for LLVM 22 in C++; hoisting conditions check loop structure, memory access, side effects, and speculative safety.</li><li>Built a Python harness comparing program behavior before/after optimization and checking transform/no-transform cases.</li></ul></div></div>                                                                                                                                                                                                                                                                                                                                                 | <div><b>Backend / Codegen</b><p>SysV x86-64, liveness/interference, register allocation, spills, phi lowering, native execution</p></div> <div><b>LLVM / IR</b><p>C++23, LLVM 22/IR, CFG/SSA, loop/interprocedural analysis, IR transformations</p></div> <div><b>Testing</b><p>assignment verifier, LLVM Verifier, structural checks, idempotence, differential execution</p></div> <div><b>Languages</b><p>C++23, Rust, C#, .NET, Linux, CMake</p></div>                                                                                 |
| PROJECTS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | EDUCATION                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <div><b>x86-64 Compiler Backend &amp; Register Allocation</b><p>SSA/CFG validation → liveness/interference → register allocation/spills → phi lowering → SysV x86-64 emission.</p><p>Assignment verifier plus native-vs-interpreter differential execution; the current project CI is green.</p></div> <div><b>LLVM Interprocedural Global IV Optimization</b><p>An LLVM 22 pass localizes supported induction-like globals after interprocedural effect analysis, affine evolution, and a separate applicability check; APInt models fixed-width wraparound.</p><p>29 positive/negative regression cases cover LLVM Verifier checks, structural expectations, idempotence, and observable before/after behavior.</p></div> <div><b>UniversalToolchain</b><p>Dependency/conflict resolution, implementation selection, pass ordering, and typed artifact routes before execution.</p><p>1,324/1,324 tests; interpreter/CIL parity and configuration checks before runtime.</p></div> | <div><b>RECOGNITION</b><div><b>LangDev'26 — Build the Language, Then Make the Abstractions Disappear</b><p>Accepted talk on UniversalToolchain's architecture; LangDev'26 takes place October 8–9, 2026 in Málaga.</p></div><div><b>UniversalToolchain — Baltic Science and Engineering Competition</b><p>First Degree Diploma and Grand Prize "Perfection as Hope" for UniversalToolchain.</p></div><div><b>MEPhI Junior</b><p>Overall winner; First Degree Diploma, 96/100 for UniversalToolchain.</p><p>Moscow / remote</p></div></div> |